AgPlane: Letting Agents Harness Their Own System-Level Behavior with eBPF

Paper #XXX
Anonymous Submission

Abstract

AI agents operate through harnesses that maintain execution, mediate tools, and apply behavioral constraints for effective and safe operation. In practice, projects often specify these constraints as natural-language harness instructions: our empirical study of 64 projects finds that 64% of instruction-file statements are behavioral directives, e.g. do not leak secrets, run tests before committing. A central component of such a harness is a runtime policy engine that enforces these constraints over the agent's concrete actions. Yet existing policy engines in agent harnesses fail to connect AI agent intent to deterministic system-level actions, leaving a *semantic gap* and making it difficult to ensure compliance or follow rules. While 83% of directives involve system-level behavior, prompt constraints operate at the probabilistic intent level. 81% of projects require cross-event state tracking, but tool-call guardrails lose track of system-level actions. 74% of system-level rules require intent-level project or task context to evaluate, while current OS-level mechanisms expect predefined static policy and return only system events without semantic context. To address this, we argue the policy engine should be exposed as a programmable interface, allowing agents to dynamically define and adapt their policy for their own system-level actions based on their intent context. We introduce AgPlane, a policy engine that allows agents to declare cross-event behavior as labeled information-flow control (IFC) policies under a layered policy model, observing and enforcing the agents' actions across process, file, and network boundaries with semantic feedback. We implement AgPlane with eBPF and evaluate it across empirical policies, system workloads, and coding tasks, showing improved policy compliance with low end-to-end overhead.

CCS Concepts: • **Computer systems organization** → *Embedded and cyber-physical systems*; • **Security and privacy** → *Software and application security*; • **Software and its engineering** → *Software safety*.

Keywords: AI agents, eBPF, BPF-LSM, labeled information-flow control, information-flow control, provenance, semantic feedback

ACM Reference Format:
Paper #XXX. 2026. AgPlane: Letting Agents Harness Their Own System-Level Behavior with eBPF. In *Proceedings of 2026 ACM SIGOPS Annual Technical Conference (ATC '26).* ACM, New York, NY, USA, 13 pages.

1 Introduction

AI agents increasingly operate as long-running actors with access to tools, files, shells, browsers, package managers, and external APIs [9, 16, 24, 53]. Coding agents are a prominent example [47, 50]. As agents gain autonomy, they require *AI agent harnesses*: software around the model that improves agent performance while enforcing behavioral constraints for safety, security, and compliance [5, 45].

A central component of such a harness is a runtime policy engine: the part that enforces behavioral constraints over the agent's concrete actions. In practice, projects encode many such constraints as behavioral policies in instruction files (`CLAUDE.md`, `AGENTS.md`) [8, 29], so harnesses can provide these instructions to the model. Because agents comply with instructions probabilistically [23, 28, 37] and can violate them through planning errors, shell-script side effects, prompt-injection drift, and opaque tool behavior [16, 21, 53], harnesses also require runtime policy mechanisms to enforce compliance [14, 15, 38, 41, 46].

Many of these policies constrain system-level actions. Our empirical study of 64 projects (§3) finds that 64% of instruction-file statements are behavioral directives, and 83% of those involve system-level behavior: commands executed, files accessed, and network connections made.

Yet existing policy engines in agent harnesses leave a *semantic gap*: *AI agent intent and directives* are expressed in natural language with project or task context, but enforcement must decide over *system actions*: concrete commands, file operations, network connections, and subprocess effects. Runtime prompt, agent-action, and tool-call policy systems [14, 15, 38, 41, 46, 49] lose track of system actions when agents shell out or spawn subprocesses: the same `git push` can be reached through a tool call, a `bash -c` wrapper, or a compiled binary; and at the repository level, 81% of projects contain cross-event directives that tool-level interception cannot enforce. OS-level mechanisms [6, 10, 13, 26, 33, 44] observe all system actions but expect pre-defined static policy and return only system events without semantic context; yet 74% of system-level directives require project or task context that is absent from low-level OS events.

We argue that an agent-harness policy engine should be programmable by the agent itself and take effect at OS level: it should let the agent use live project and task context to enforce its own actions. This creates three challenges. First, policy specification must be *expressive* enough to capture

directive intent yet *concrete* enough to compile to deterministic kernel checks. Second, enforcement must track policy-relevant state across tools, subprocesses, files, and network endpoints without imposing prohibitive per-syscall overhead. Third, programmability must not become an authority bypass: agents must be able to add context-specific constraints, but neither they nor their sub-agents may weaken inherited policy.

AgPlane is a programmable OS-level runtime policy enforcement system for AI agent harnesses. Agents express behavioral policies in a compact domain-specific language (DSL) generated from natural-language project directives; AgPlane compiles them into labeled information-flow rules and loads them into an eBPF/BPF-LSM (Linux Security Modules) enforcement engine. Named labels propagate across processes, files, and network endpoints at kernel hooks, encoding execution history as checkable state. When a labeled event matches a rule, AgPlane returns semantic feedback: which rule matched, why, and what compliant alternative satisfies it. Each rule independently selects one of three effects: notify (observe), block (pre-operation denial), or kill (post-operation termination). A layered authority model ensures that higher-authority rules (e.g., platform-level "do not leak secrets") cannot be weakened by agent-authored policy.

We make three contributions:

1. An **empirical study** of instruction files from 64 projects, showing that cross-event system-level directives are common (81% of repositories) and that 74% of system-level directives require project or task context absent from low-level OS events (§3).
2. **AgPlane**, a programmable OS-level runtime policy enforcement system for AI agent harnesses. AgPlane compiles behavioral policies into eBPF/BPF-LSM labeled information-flow rules, enforces them across process, file, and network boundaries, and returns semantic feedback that explains rule matches in terms of AI agent intent and concrete remediation (§4–5).
3. An **evaluation** across directive-derived rules, system workloads, and a 20-task OctoBench subset. On a 190-trace decision-compliance benchmark sampled from 38 directives, AgPlane reaches a 75.8% decision-compliance rate (DCR), 27.4 percentage points higher than prompt-filter and 30.5 points higher than tool-regex, while adding less than 2% end-to-end overhead on agent workloads (§6).

2 Background

AI agents and harnesses. AI agents combine model reasoning with external tools and long-running state. They may browse untrusted content, call APIs, manipulate files, execute code, or coordinate workflows across multiple steps [9, 16, 53]. Coding agents such as Claude Code [1], Codex CLI [32], Cursor [2], Devin [12], SWE-agent [50], and OpenHands [47] are a representative deployment domain where these effects are readily observable. Agents operate through an *AI agent harness*: software around the model that maintains the agent loop and session state, routes tool calls, mediates shell, file, and network access, and returns results or feedback to the model, improving agent performance while enforcing behavioral constraints [5, 45]. A single tool invocation can run arbitrary scripts, spawn subprocesses, and touch many files and endpoints before control returns to the model. Projects encode behavioral expectations in instruction files (`CLAUDE.md`, `AGENTS.md`) [7, 8, 29, 39] as *AI agent intent*: natural-language constraints over what the agent should do, avoid, or condition on project and task context. The harness must enforce that intent over *system actions*: concrete commands, file operations, network connections, and subprocess effects.

Information-flow control. Information-flow control (IFC) tracks how data or authority moves from sources to sinks over time. Static IFC systems embed flow constraints in type systems or language annotations [31]. Dynamic IFC and provenance systems attach labels to OS objects and propagate them at runtime: when a process reads a labeled file, the process acquires that label; when it forks, the child inherits it [11, 18, 34, 40]. Later operations are checked against the accumulated label state. OS-level IFC systems such as Flume [26] and HiStar [52] enforce flow policies across processes and files in the kernel, while whole-system provenance frameworks such as PASS [30], Hi-Fi [36], LPM [4], CamFlow [34], and CamQuery [33] record and query cross-event label histories. IFC thus captures constraints whose correctness depends on execution history, which is the same pattern that agent behavioral directives ("run tests before committing") exhibit. More general OS-level mechanisms (seccomp [13], AppArmor [6], Capsicum [48], Landlock [44], BPF-LSM [43], Tetragon [10], KubeArmor [27], and BPF-Contain [19]) also observe or restrict system effects below the tool layer, but require statically pre-written policies and return opaque errors with no connection to the agent's declared directives.

AI agent policy enforcement. We use *runtime policy enforcement system* for the part of an AI agent harness that enforces safety, security, and compliance constraints while the agent executes. Existing AI agent harnesses place such enforcement at one of three levels. Prompt instructions rely on the model's own compliance, which is inherently probabilistic [23, 28, 37]. Indirect-prompt-injection benchmarks further show that untrusted tool outputs can redirect agents toward unauthorized actions [16, 21, 53]. Programmable rails and guard agents can check prompts, responses, or action trajectories at runtime [9, 38, 49]. Tool-call guardrails such as AgentSpec, Progent, and Invariant Guardrails [22, 41, 46] intercept tool-call names and arguments deterministically, while application-level IFC systems such as FIDES [14] and CaMeL [15] track data flow within the agent loop. These mechanisms observe only the AI agent harness request, not

the system-level effects that follow once a tool starts executing: a subprocess, a shell-out, or a compiled binary can bypass the tool boundary entirely. What remains missing is the connection between AI agent intent and project or task context on one side, and deterministic OS-level observation and enforcement on the other. The empirical study below characterizes how strongly real project instructions demand this connection.

3 Empirical Study

We analyze real agent instruction files to answer a prerequisite question: do project instructions contain behavioral policies that demand OS-level, cross-event enforcement? This section characterizes the directives, classifies their enforcement requirements, and quantifies the context they need to become concrete rules.

3.1 Methodology

We collected public GitHub repositories containing `CLAUDE.md` or `AGENTS.md`, prioritizing the AI-agent ecosystem over filename-only matches and excluding non-code, inactive, fake-star, and stub repositories. The snapshot (2026-05-23 UTC) contains 64 repositories, 84 deduplicated instruction files, and 2,116 extracted statements. A *statement* is a coherent unit expressing one claim, directive, or constraint. A two-pass LLM-assisted pipeline extracted statements with source line ranges and four labels (content type, topic, enforcement level, context requirement); a validation script verified full source coverage and verbatim span matching. A stratified sample of 100 statements was independently reviewed by human annotators, with enforceability labels additionally cross-checked by independent Claude and Codex agents. The following subsections report results along the four labeling axes; E-RQ3 identifies the OS-enforceable subset used in the evaluation (§6). Table 1 collects representative statements that illustrate all four axes; the labels **S1**–**S8** are referenced throughout.

3.2 E-RQ1: Are Instruction Files Behavioral Policies?

A statement is a *directive* if it asks the agent to perform, avoid, or condition an action; otherwise it is descriptive context. Of the 2,116 statements, 1,361 (64.3%) are directives and 755 (35.7%) are descriptions (Figure 1).

Instruction files are predominantly behavioral policies: the median repository has 71% directives. 75% of repositories have a directive fraction above 50%. At the extremes, one repository is entirely descriptive (0% directives) while another is 97% directives. This majority is invisible at line level: directives average 3.6 lines per statement while descriptions average 6.8 lines, so the same files appear balanced (49% directive) when measured by lines.

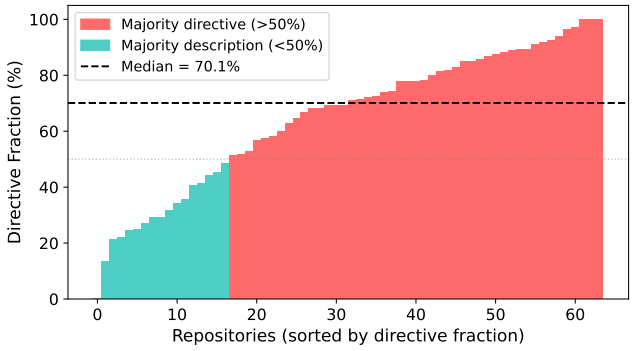

Figure 1. Directive fraction per repository by statement count. Most repositories contain a majority of directive statements.

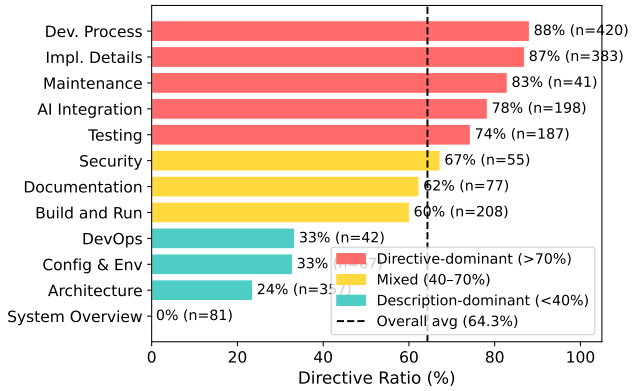

Figure 2. Directive ratio by topic.

3.3 E-RQ2: Which Topics Contain Directives?

Each statement is assigned to one of 12 topic categories adapted from prior instruction-file studies [7], applied at statement granularity rather than file granularity (Figure 2).

Directive density ranges from 0% (System Overview) to 87% (Development Process). Development Process and Implementation Details are directive-heavy (86.7% and 85.2%, respectively). Architecture is mostly descriptive: directory layouts and design summaries make it one of the largest topics by line count, yet only 23.0% of its statements are directives. A file classified as "Architecture" by prior studies may mostly describe the project, while a shorter Development Process section packs many enforceable rules.

3.4 E-RQ3: Which Directives Require OS-Level Enforcement?

Each directive is classified into the first matching tier of an enforcement waterfall (Figure 3): *semantic-only* (reasoning, communication, or output style), *content* (predicates over file contents), *per-event* (a single command, file access, or network connection), and *cross-event* (history, ordering, lineage,

Table 1. Representative corpus statements illustrating all four classification axes. Enforcement level and context requirement apply only to directives (— = not applicable).

	Statement	Type	Topic	Enforcement	Context
S1	"The backend uses Express with TypeScript."	Desc	Architecture	—	—
S2	"Always explain your reasoning before changes."	Dir	AI Integr.	Semantic	—
S3	"Prefer `const` over `let`."	Dir	Impl. Det.	Content	None
S4	"Never push to main directly."	Dir	Dev. Proc.	Per-event	None
S5	"Never modify upstream source code."	Dir	Dev. Proc.	Per-event	Project
S6	"Run the full test suite before committing."	Dir	Testing	Cross-event	Project
S7	"Data read from `.env` must not reach the network."	Dir	Security	Cross-event	Project
S8	"Do not update dependencies without approval."	Dir	Maint.	Per-event	Task

or information flow). We call the union of content, per-event, and cross-event directives *system-observable*; the per-event and cross-event subset that AgPlane can enforce at OS hooks is *OS-enforceable*.

83% of directives constrain system-level actions. Of 1,361 directives, 234 (17.2%) are semantic-only, 520 (38.2%) require content inspection, 392 (28.8%) match a single OS event, and 215 (15.8%) require cross-event state (Table 1, S2–S7 illustrate all four levels). Content inspection alone covers 55.4% of directives; adding per-event matching reaches 84.2%; the remaining 15.8% require cross-event IFC.

Cross-event directives concentrate in Development Process, which accounts for 39.5% of all cross-event directives (Figure 4). Four recurring patterns emerge: temporal ordering ("run tests before committing"), cross-file consistency ("update docs when behavior changes"), multi-step workflows (release checklists with verification steps), and conditional triggers ("if you change specs, also update SDK"). These map directly to IFC primitives: lineage gates, label propagation, and staleness-aware re-arming.

At the repository level, 81% of repositories contain at least one cross-event directive, and 43% require all four enforcement layers.

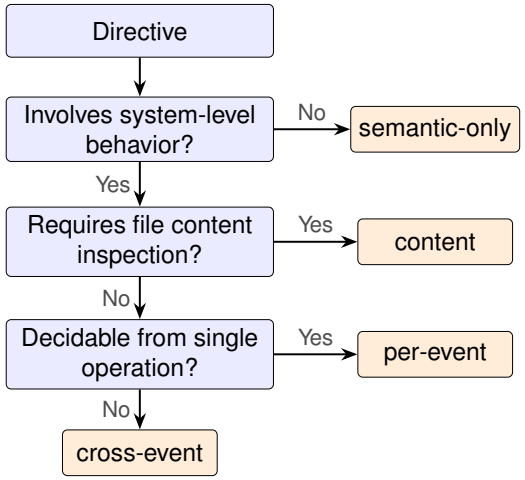

Figure 3. Enforcement-level waterfall. Each directive exits at the first matching tier.

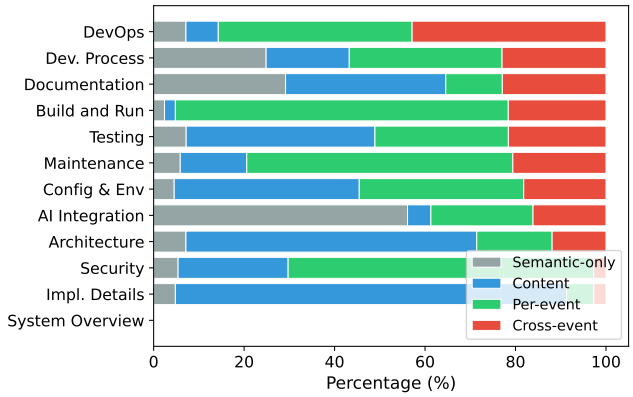

Figure 4. Enforcement profile by topic (normalized). Topics exhibit distinct archetypes; cross-event directives concentrate in Development Process.

3.5 E-RQ4: What Context Is Needed to Instantiate Rules?

Each system-observable directive is classified by what additional context an enforcement mechanism needs beyond the directive text itself (Figure 5): *self-contained* if all commands, paths, and patterns are explicit; *project context* if it references an unresolved repository-specific concept ("the full test suite," "the migration tool"); or *task context* if it depends on the current user request or a per-session grant ("unless explicitly requested," "without approval").

74% of system-observable directives require project or task context to instantiate as concrete rules. Of the 1,127 system-observable directives, only 26.4% are self-contained. Another 64.2% require project context: the directive mentions "the test suite," "the linter," or "upstream source," whose concrete command or path must be resolved from the repository. The remaining 9.4% require task context (Figure 6;

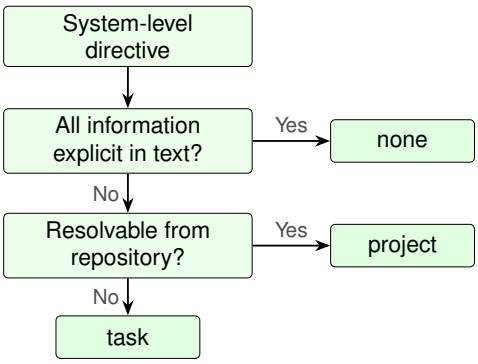

Figure 5. Context-requirement waterfall. Each system-level directive exits at the first matching tier.

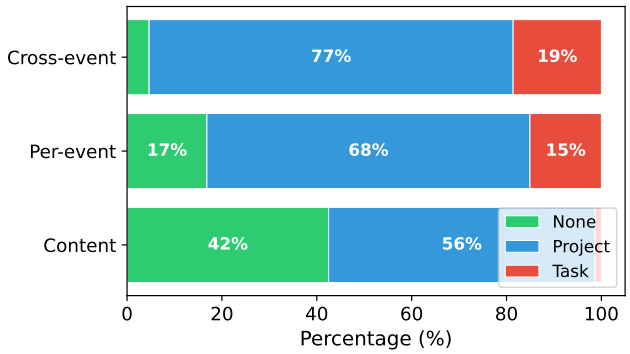

Figure 6. Context requirement by enforcement level. Cross-event directives are 95% context-dependent; content directives are 42% self-contained.

Table 1: S4 is self-contained; S5–S7 require project context; S8 requires task context).

Cross-event directives are 95% context-dependent, compared to 58% for content directives. Of cross-event directives, 76.7% require project context and 18.6% task context (95.3% total); content directives are only 57.5% context-dependent. This compounds the enforcement challenge: directives that require cross-event IFC almost never specify the concrete commands and paths needed to write the rule. Statically authored rules can cover only the self-contained fraction; the majority require the agent itself to read the repository, interpret the current task, and produce a concrete policy before enforcement begins.

4 Design

The empirical study establishes one overarching requirement: connect AI agent intent and project or task context to deterministic OS-level enforcement. Achieving this raises three design challenges.

C1: Expressive yet enforceable policy specification (§4.3). Instruction-file directives are written over repository and task concepts such as "the test suite," "approved endpoints," or "do not push to main." Kernel enforcement operates over primitive operations: exec, open, write, and connect events with paths, arguments, processes, and endpoints. A specification that stays at the directive level is too ambiguous to enforce deterministically; one written directly at the syscall level is verbose, brittle, and exposes kernel internals that agents and operators cannot reliably audit. The DSL must be *concrete* enough to lower to fixed-size kernel predicates, yet *expressive* enough to preserve the directive's scope, context, and rationale.

C2: Efficient cross-event enforcement (§4.4). 81% of repositories require policies that span multiple operations: "run tests before committing" depends on whether a test execution occurred after the last source change. No single event reveals the full behavior, yet enforcement must avoid reconstructing or querying complete execution traces on every syscall. The enforcement layer must maintain sufficient policy-relevant history across tools, subprocesses, files, and network endpoints for deterministic checks without imposing prohibitive per-syscall overhead.

C3: Authority-safe agent programmability (§4.5). Agents must be able to adapt policy at runtime: a sub-agent may need a narrower scope, or the current task may require a sanctioned exception to an inherited constraint. Letting the constrained actor write or specialize policy introduces a privilege-weakening risk: the agent must be able to add task-specific constraints, but this programmability must not become an authority bypass. Agents and sub-agents may add rules, specialize targets, or narrow scope, but must never weaken platform or project constraints.

4.1 Threat Model and Assumptions

AgPlane targets a *cooperative but unreliable* agent. The agent's original intent is honest: it aims to follow project constraints. But execution may diverge due to planning errors, shell-script side effects, opaque tool behavior, or prompt injection [35] that causes the agent to act against its declared policy. Higher-authority policies (platform, project) are authored by trusted principals and cannot be weakened by the agent; they guard against both unreliable execution and hijacked agent-level declarations. The system does not target a malicious process that attacks the kernel, exploits the loader, or deliberately evades policy through adversarial obfuscation.

The trusted computing base (TCB) consists of the compiler, the eBPF enforcement engine, the runner, and higher-authority policies. The agent's entire process tree, including tools, shells, subprocesses, and sub-agents, constitutes the monitored domain. Kernel compromise, root or `CAP_BPF` compromise, deliberate side channels, and semantic content checks not observable at syscall boundaries are out of scope.

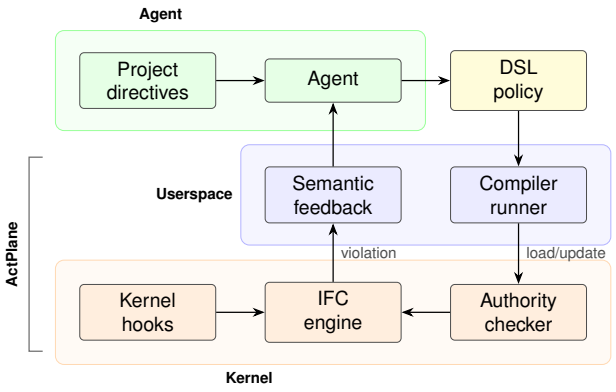

Figure 7. AgPlane architecture: policy generation, compilation, kernel enforcement, and feedback loop.

4.2 System Overview

AgPlane addresses each challenge with one abstraction: a policy DSL (C1) that captures directives at an intermediate granularity and compiles them into kernel-level IFC tables; a labeled information-flow model (C2) that summarizes execution history as compact object labels; and a layered authority model (C3) that allows runtime policy adaptation while ensuring monotonic tightening. When a rule match occurs, AgPlane returns semantic feedback to the agent: the rule name, reason, and remediation. Figure 7 shows the end-to-end pipeline.

4.3 Policy DSL

To address C1, the DSL bridges AI agent intent and enforceable system actions. A policy contains *sources* that introduce labels, *rules* that match labeled events at sinks, and optional *gates* that express cross-event ordering constraints. The DSL restricts expressiveness to source/sink/gate patterns that compile to fixed-size rodata, avoiding direct exposure of BPF hooks, maps, label bits, or verifier constraints.

Sources bind labels to system objects: an `exec` source labels processes that execute a matching binary; a `file` source labels matching paths; a network source labels matching endpoints. Rules bind an effect (`notify`, `block`, or `kill`) to an operation (`exec`, `open`, `read`, `write`, `unlink`, or `connect`). Conditions express common agent workflow constraints: a target allow-list, a lineage gate, or a temporal `after ... since ...` ordering gate.

Figure 8 illustrates the three main rule shapes. The first is per-event: if the agent process pushes to `main`, the event matches immediately. The second is cross-event: committing is compliant only if the required test command ran after the most recent source write. The third uses the same temporal gate for authority exceptions: force-pushing is blocked unless the agent first writes a declaration file, which higher-authority policy can further constrain.

```
source AGENT = exec "claude"
source SECRET = file "**/.env"

# Per-event: from CoplayDev/unity-mcp
rule no-push-main:
  kill exec "git" "push" "main" if AGENT
 because "Do not push to main; open a PR instead."

# Cross-event: from chenhg5/cc-connect
rule tests-before-commit:
  block exec "git" "commit"
    if AGENT unless after exec "go"
      since write "**/*.go"
 because "Run `go test ./...` before committing."

# Authority exception gate
rule no-force-push:
  block exec "git" "push" "--force"
    if AGENT unless after write
      ".agent/force-push-necessary"
 because "Write a justification before force-pushing."
```

Figure 8. Three AgPlane rules drawn from real projects: a per-event rule, a cross-event ordering gate, and an authority exception gate.

The DSL is intentionally narrower than a general mandatory access-control language. It targets the policy shapes that recur in agent instruction files: prohibit a command, restrict writes to a scope, block a sink after a sensitive read, require a validation step before a release action, and route sensitive data through an approved tool. The compiler lowers these patterns into fixed-size kernel tables; policy authors need not reason about BPF maps, verifier constraints, or binary layout.

4.4 Labeled Information-Flow Model

To address C2, AgPlane represents policy state as labels on OS objects. Processes, files, and network endpoints each carry a 64-bit label mask, enabling cross-boundary flow policies (e.g., "data read from `.env` must not reach a network endpoint"). A source declaration adds labels when an object matches the source pattern. Let $L(x)$ denote the label mask of object x. Labels propagate through observed system-level edges:

- **fork**$(p \to c)$: $L(c) := L(c) \cup L(p)$.
- **exec**(p, f): $L(p) := L(p) \cup L(f) \cup S(f)$, where $S(f)$ is the set of source labels matching f.
- **read**(p, f): $L(p) := L(p) \cup L(f)$.
- **write**(p, f): $L(f) := L(f) \cup L(p)$.
- **connect**(p, e): $L(e) := L(e) \cup L(p)$.

These transfer functions maintain a key invariant: if an object carries label ℓ, then information from a source tagged

with ℓ has reached that object through observed execution edges. Rules check whether an event's subject or target carries required labels, carries forbidden labels, or satisfies a temporal gate.

The model operates at object granularity rather than byte granularity, trading precision for low-overhead whole-session coverage. This is a deliberate fit for AI agent harnesses: repository instructions refer to commands, files, directories, endpoints, and workflow steps, not individual bytes. Object-level labels capture cross-event compliance patterns that tool-call filters miss, without incurring the complexity of byte-level taint tracking [25].

For efficiency, labels summarize history rather than recording traces. Each object stores a fixed-size mask, and each rule checks masks and bounded predicates at the current event. This keeps cross-event enforcement on the syscall path without log reconstruction or unbounded history queries.

Labels are monotonic: propagation adds labels but never removes them. Once an object carries a label, every subsequent consumer inherits it. This ensures that cross-event constraints remain checkable for the entire session without requiring explicit label cleanup.

4.5 Layered Policy Authority

To address C3, agents must adapt policy at runtime: a sub-agent spawned for a narrow sub-task should operate under tighter constraints, and the current task may require a sanctioned exception to an inherited constraint; but adaptation must never weaken platform or project constraints.

AgPlane organizes policy around a *domain tree*. A domain is the runtime policy boundary for a process tree: every pid maps to a domain, and every domain maps to an effective policy set. The core data model is:

```
pid    -> domain
domain -> parent + policy(domain)
policy(D) = policy(parent(D)) + local(D)
```

Policy evolution is *monotonically tightening*: a child domain inherits all parent rules and may add local rules, labels, or gates, but cannot remove, disable, or weaken any inherited rule. When an agent forks a sub-agent, the child process enters a child domain that inherits the parent's full policy; the sub-agent may add further restrictions but cannot shed inherited constraints.

The root domains correspond to platform/operator and project-maintainer policy; the agent creates session and sub-agent domains below them. Higher-authority rules that admit exceptions use the same temporal gate as cross-event workflow rules (Figure 8, `no-force-push`): the agent satisfies the gate by creating a declaration file, which the IFC engine tracks like any other label-propagating event. Higher-authority policy can further constrain which processes may write the declaration file, keeping exception paths under platform or project authority.

The prototype resolves the initial domain tree at compile or load time. Runtime policy additions (sub-agent restrictions, task-specific rules) are submitted as precompiled deltas through a userspace ring buffer. The kernel authority checker admits a delta only if it is additive: it may bind new rules, add labels, or narrow scope, but it cannot remove inherited rules or widen scope. Accepted deltas are merged into the domain's effective policy; the kernel does not parse configuration or resolve project context on the event path.

5 Implementation

AgPlane consists of a Rust compiler/runner (3.2 K LoC) and an eBPF enforcement engine (1.8 K LoC of BPF C). The compiler parses the DSL, allocates label bits, lowers Boolean label expressions and path/network patterns, and emits a fixed-size `#[repr(C)]` configuration blob consumed directly by the eBPF loader. Human-readable rule names and reasons stay in userspace metadata; the kernel receives only compact rule tables and emits rule identifiers when events match.

The eBPF engine attaches to process, file, and network hooks via BPF-LSM (pre-operation enforcement) and tracepoints (observation and post-operation termination). It maintains label state in per-object BPF maps, applies the transfer functions from §4, and evaluates the compiled rules on each observed event. Userspace does not re-detect violations: it loads the policy, starts the target process, and formats kernel-reported matches into semantic feedback.

The runner (`AgPlane run – <cmd>`) discovers the project policy file, compiles and loads it before the target starts, seeds the target process tree with the agent label, and records kernel matches in a feedback file. Compatible agent hooks relay this feedback into the model's next turn. The runner is intentionally thin: policy decisions remain in the kernel; the runner mediates between the compiled policy, the target process, and agent-facing diagnostics.

6 Evaluation

We evaluate AgPlane along three evaluation research questions (RQ1–RQ3, distinct from the empirical E-RQs in §3). RQ1 measures end-to-end policy compliance on directive-derived rules from the empirical corpus; RQ2 quantifies per-event and end-to-end overhead; RQ3 tests real-world coding tasks on an OctoBench subset with the official judge.

RQ1 (Policy compliance) Compared with prompt-filter, tool-layer, and feedback-ablation baselines, does AgPlane improve policy compliance for directive-derived rules across direct and bypass execution paths?

RQ2 (Overhead) What is the per-event and end-to-end overhead of AgPlane's label propagation and rule checking?

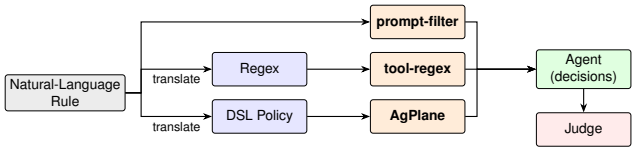

Figure 9. RQ1 evaluation pipeline: three enforcement paths from natural-language rule to agent-level decision.

RQ3 (Real-world coding tasks) On complete coding-agent tasks in real repositories, does AgPlane's OS-level enforcement and semantic feedback improve policy compliance and task success rate?

6.1 Experimental Setup

Hardware and kernel. All experiments run on a single machine with an Intel Core Ultra 9 285K CPU (24 hardware cores), 125 GiB RAM, Linux 6.15.11, ext4 filesystem, and libbpf 1.x with `bpf_loop` support. The live enforcement runs require BPF-LSM active (`bpf` in `/sys/kernel/security/lsm`); the RQ2 overhead artifacts record the exact LSM state used for each performance run.

Agent environment. The active AI agent harnesses are Claude Code (via the AgPlane feedback hook) and OpenAI Codex CLI (via tool-error feedback). The tested LLM for RQ1 trace replay is Qwen-2.5-27B; the translation agent uses GPT-5.5 to generate policy artifacts. The trace generator and trajectory judge each use Claude Sonnet. Harness versions are pinned in the artifact metadata.

Compared systems. RQ1 uses four runtime policy enforcement systems on the same trace-conditioned AI agent harness: a prompt-based action filter, a tool-input regex policy, AgPlane without structured corrective feedback, and full AgPlane. The prompt and regex baselines instantiate two common runtime enforcement layers: LLM/prompt-based rails over proposed actions [38, 49] and deterministic tool-call policies over tool names and arguments in the style of AgentSpec [46]. The opaque ablation isolates the value of semantic feedback on top of OS-level observation.

6.2 RQ1: End-to-End Policy Compliance

Motivation for a new benchmark. Existing enforcement benchmarks such as ShieldAgent-Bench [**?**] and ToolSafe [**?**] evaluate guardrail precision and recall at the tool-call or trajectory level. Runtime enforcement evaluations in AgentSpec, Progent, FIDES, and CaMeL [14, 15, 41, 46] measure attack-success-rate reduction on AgentDojo scenarios [16]. However, all operate at the tool-call boundary: they do not test whether enforcement holds when an agent reaches the same system effect through a subprocess, a shell script, or a compiled binary.

Rule set and scope. The empirical study identifies 607 OS-enforceable directives: 392 per-event and 215 cross-event. RQ1 uses this subset because these directives can be represented at process, file, or network hooks. Semantic-only and content directives are excluded: the former have no system-observable counterpart; the latter require file-content inspection rather than OS-level IFC. We evaluate 38 manifest-selected statements, sampling 2–3 from each of 15 repositories to cover both per-event and cross-event rules, project- and task-context requirements, and common repository policy themes. The final manifest contains 20 per-event and 18 cross-event rules, proportional to the 65%/35% split in the full corpus. No system outcome is used to select statements or repair policy artifacts.

Pipeline. We construct a trace-conditioned benchmark that exercises five trace families, including script-mediated and opaque-fixture violations that are invisible at the tool-call level. The pipeline has five stages with strict actor separation (Figure 9). A *translation agent* reads the sampled directive text, the project's `CLAUDE.md`, the cloned repository, and the DSL reference to produce a rule file. The agent iteratively invokes the AgPlane compiler as a tool: if a generated rule fails to compile, the agent observes the compiler diagnostic, revises the rule, and retries until compilation succeeds. The translation agent cannot see traces or ground truth. For prompt-filter and tool-regex, a translation agent generates the corresponding policy artifact (LLM classifier prompt or regex) from the same directive and project context; all artifacts are frozen before execution. A separate *trace generator* reads the directive and project structure to produce five trace families per statement, each with a ground-truth header; it cannot see the rule file, preventing circular bias. The families cover allowed-effect compliant behavior, lookalike compliant behavior, directly visible violations, script-mediated violations, and opaque-fixture violations. Script-mediated traces split script authoring and execution across separate tool calls; opaque traces expose only a neutral helper entrypoint while the real side effect is visible at runtime. Each trace is a JSONL transcript containing a user prompt and concrete tool calls that establish the repo state. During replay the harness executes each tool call on a copy-on-write repository clone, and the tested agent continues for a bounded number of real tool steps. Ground truth and policy artifacts are not shown to the tested agent. For each statement we construct five traces: two compliant families (allowed-effect and lookalike) and three violation families (visible, script-mediated, and opaque-fixture). This yields 190 traces and 760 system–trace cells across four systems: prompt-filter, tool-regex, AgPlane without structured feedback (AgPlane-opaque), and full AgPlane. Figure 8 shows representative AgPlane rules.

Metric. A separate LLM trajectory judge assigns TP, TN, FP, or FN from the raw runner result. TP means a violation trace received enforcement-visible intervention or agent

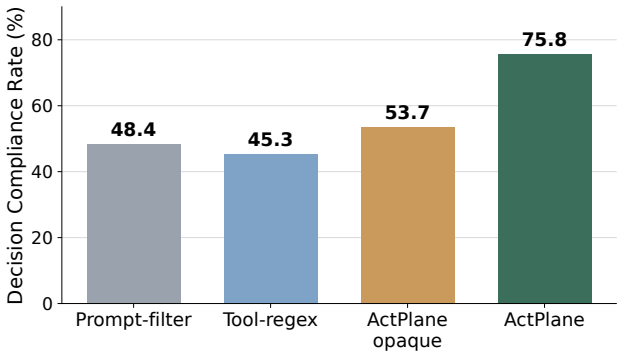

Figure 10. Overall RQ1 Decision Compliance Rate across 190 traces.

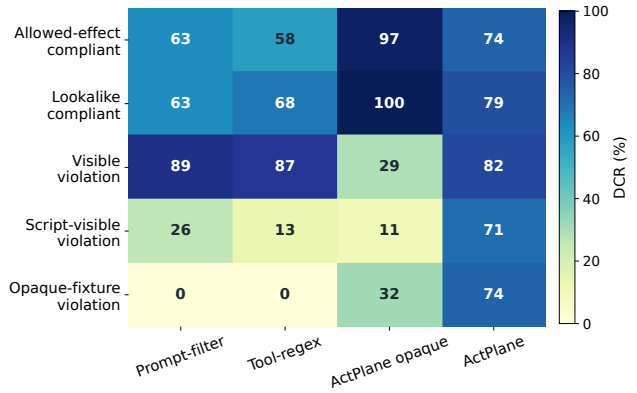

Figure 11. RQ1 breakdown by trace family. Cells show DCR (%) for each system×family; darker is higher.

Table 2. RQ1 confusion matrix by runtime policy enforcement system.

System	DCR	TP	TN	FP	FN	Judged
Prompt-filter	48.4%	44	48	28	70	190
Tool-regex	45.3%	38	48	28	76	190
AgPlane-opaque	53.7%	27	**75**	**1**	87	190
AgPlane	**75.8%**	**86**	58	18	**28**	190

awareness; TN means a compliant trace completed without wrongful intervention; FP means a compliant trace was wrongly blocked or steered; FN means a violation trace was missed. We report *Decision Compliance Rate* (DCR), the fraction of trace-conditioned decision points whose final outcome is policy-correct: DCR = (TP + TN)/(TP + TN + FP + FN). DCR is end-to-end: it includes policy translation, runtime enforcement, feedback delivery, and agent recovery. All 760 system–trace cells and all 760 trajectory-judge cells are present, with no judge parse errors.

Results. AgPlane reaches a DCR 27.4 percentage points higher than prompt-filter and 30.5 points higher than tool-regex. The difference comes primarily from violation recall. Full AgPlane reduces FNs to 28, compared with 70 for prompt-filter and 76 for tool-regex, because it observes runtime side effects below the tool layer and can detect script- and fixture-mediated violations that are not available to prompt or raw tool-input filters. It also reduces false positives on compliant traces relative to the baselines, though this margin is smaller: full AgPlane has 18 FPs, compared with 28 for both prompt-filter and tool-regex.

The feedback ablation isolates the value of semantic recovery information. AgPlane-opaque enforces the same kernel-level rules but returns no structured feedback: the agent sees only a raw `EPERM` or process termination. Without a feedback channel, the system never actively communicates with the agent on compliant traces, which explains its near-zero FP (1) and high TN (75), not because enforcement is more precise, but because the agent receives no enforcement signal on compliant traces. Conversely, on violation traces the kernel does block the offending syscall, yet the agent cannot diagnose the cause or recover, so the judge classifies most blocked violations as FN (87 FNs). Full AgPlane opens a structured feedback channel: rule name, reason, and remediation reach the agent after every match. This raises violation recall substantially (FN drops from 87 to 28, TP rises from 27 to 86) but introduces a tradeoff on compliant traces: the active feedback channel occasionally triggers agent over-correction on boundary cases, raising FP from 1 to 18 and lowering TN from 75 to 58. The net effect is positive (correct responses rise from 102 to 144) because the mechanism is not additional detection alone; the feedback channel converts kernel-level observation into agent-level recovery.

Figure 11 breaks down DCR by trace family. On the two compliant families (allowed-effect and lookalike), all systems except AgPlane-opaque show comparable performance; AgPlane-opaque's near-zero FP rate gives it the highest compliant-trace DCR. The difference emerges on violation families. On directly visible violations, prompt-filter and tool-regex detect some cases, but AgPlane detects substantially more because it observes the actual system effect rather than the proposed tool input. On script-mediated and opaque-fixture violations, prompt-filter and tool-regex largely fail: tool-call interception cannot see effects that occur inside a subprocess or behind a neutral entrypoint. AgPlane retains detection through kernel-level label propagation, which follows the effect regardless of how the agent invoked it. The feedback ablation confirms that detection alone is insufficient: AgPlane-opaque observes the same kernel events but produces low violation-family DCR because the agent cannot recover without a structured corrective payload.

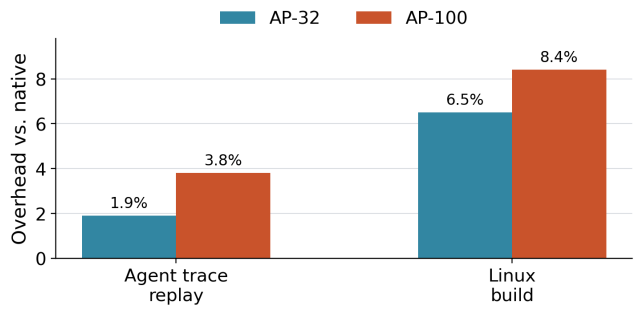

Figure 12. End-to-end overhead normalized to native execution.

6.3 RQ2: Overhead

6.3.1 Macrobenchmarks (End-to-End Workloads). We measure end-to-end overhead on two deterministic workloads under no-hit AgPlane configurations: (1) an agent trace suite that replays 20 corpus-derived traces (68 tool actions, 20 Bash subprocesses), and (2) a Linux kernel build (`defconfig` + `vmlinux`, `make -j24`, clean output directory). Each workload runs three trials per configuration. Fixed workloads eliminate LLM inference variance and isolate AgPlane's runtime cost.

Results. At 32 active no-hit rules, AgPlane adds 1.9% overhead on the agent-trace replay and 6.5% on the Linux kernel build. At 100 rules, overhead remains below 8.4%. The agent-trace workload exhibits lower overhead because tool actions are interspersed with model inference pauses that dwarf syscall cost; the kernel build stresses sustained I/O and process creation, exposing more of AgPlane's per-event cost.

6.3.2 Microbenchmarks (Per-Syscall Latency). We measure AgPlane's per-event latency for six syscall types (`fork`, `exec`, `open`, `read`, `write`, `connect`) under five no-hit configurations: native, and AgPlane with 1, 10, 32, and 100 active rules (AP-N denotes AgPlane with N rules). Each configuration × syscall type runs 10K–100K iterations pinned to a single CPU core. For each trial we compute p50 and p99 latencies; Table 3 reports the median across seven trials.

Measured operations.

- `fork`: `fork()` + parent `waitpid()`.
- `exec`: `fork()` + child `execve("/bin/true")` + parent `waitpid()`.
- `open`: `open(O_RDONLY|O_CLOEXEC)`; `close()` outside the timed region.
- `read`: `read(fd, buf, 4096)` from an already-open file.
- `write`: `write(fd, buf, 4096)` to a temp file.
- `connect`: UDP `connect(127.0.0.1:9)` on a pre-created socket.

Table 3. Per-operation latency in microseconds for no-hit configurations. Values are medians across seven trials.

Operation	Native p50	AP-32 p50	AP-100 p50	AP-100 p99
`fork`	48.94	74.05	69.33	178.01
`exec`	248.30	314.86	317.03	490.37
`open`	0.58	6.92	13.40	15.36
`read`	0.31	0.72	0.78	1.42
`write`	0.27	0.79	0.84	1.59
`connect`	0.58	1.98	3.17	3.90
Average	*49.83*	*66.55*	*67.43*	*115.11*

Results. Table 3 reports per-call latencies for the six measured operations. `fork` and `exec` dominate per-call cost in absolute terms; their overhead ratio is modest (1.3–1.5×) because process-management and scheduler latency dominates the baseline. File operations (`open`, `read`, `write`) and `connect` are fast in the native case; AgPlane adds a path-hash lookup and rule scan on each. The arithmetic average across all six operations is 67.43 µs at AP-100 p50, a 1.4× multiplier over the native average.

Despite per-call multipliers on individual fast-path operations, the macrobenchmark impact remains low (1.9%–8.4%) because the absolute added cost per call is small relative to the computation, I/O, and LLM inference time that dominates agent workloads. The cumulative AgPlane overhead of an entire tool-call's syscall sequence is 5–6 orders of magnitude smaller than a single LLM inference turn (2–10 s).

All overhead measurements use no-hit configurations, which exercise steady-state label propagation and rule scanning without violation reporting or userspace feedback formatting.

6.4 RQ3: Real-World Coding Tasks (OctoBench)

Goal. Unlike RQ1, which isolates single decision points, RQ3 evaluates AgPlane on complete coding-agent tasks in real repositories, scored by the official OctoBench checklist judge.

Benchmark and subset. OctoBench [17] is a repository-grounded coding benchmark with 217 Dockerized tasks spanning system prompts, tool schemas, project files (`CLAUDE.md`, `AGENTS.md`), and user queries. We use the official evaluator unchanged. Because AgPlane cannot observe purely semantic checks (e.g., tone), we select a 20-task OS-observable subset. A task is eligible when at least one checklist item maps to a concrete command, file operation, generated artifact, or tool-execution constraint that can be expressed as a case-specific policy before execution. The selected subset spans `CLAUDE.md`, `AGENTS.md`, and user-query policy sources, two agent scaffolds, and seven Docker images. Pure style, tone, and natural-language-only cases are excluded; the remaining 197 tasks lack an OS-observable checklist item (e.g., "respond

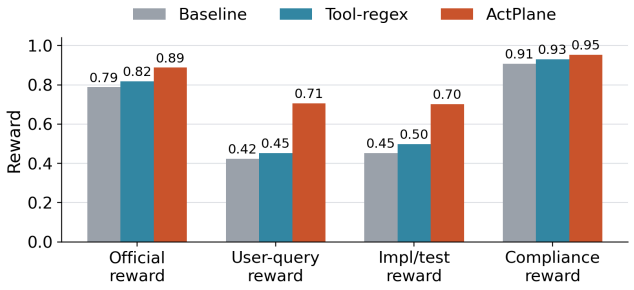

Figure 13. OctoBench 20-task subset: reward breakdown by system.

in a friendly tone") and fall outside AgPlane's enforcement scope. Each task runs under three conditions: baseline (no enforcement), tool-regex (case-specific tool-call hooks), and AgPlane (OS-level hooks plus semantic feedback).

Metrics. The primary metric is official OctoBench reward (fraction of checklist policies judged satisfied). We additionally report three diagnostic submetrics from the same checklist results: *user-query reward* (user-task checks only), *implementation/test reward* (implementation, testing, configuration, and modification checks), and *compliance reward* (compliance-typed checks). All submetrics use the official LLM-based checklist judge.

Results. On the 20-task subset, official reward is 0.788 (baseline), 0.818 (tool-regex), and 0.888 (AgPlane): a 10.0-point gain over baseline and 7.0 points over tool-regex. The improvement is not limited to compliance-typed checks: user-query reward rises by 28.2 points and implementation/test reward by 25.0 points over baseline. These results support the claim that OS-level enforcement with semantic feedback improves policy compliance on real coding-agent tasks. We note that OctoBench uses an LLM checklist judge rather than deterministic test scripts, so we do not claim deterministic task completion.

7 Related Work

In-kernel provenance and information flow. Whole-system provenance systems such as PASS [30], Hi-Fi [36], LPM [4], and CamFlow [34] record audit-grade provenance graphs via LSM hooks but do not compile agent-oriented policy or return semantic feedback. CamQuery [33] is the closest prior system: it evaluates provenance queries inline in the kernel, propagates labels across OS objects, and can deny matching operations. AgPlane shares this label-propagation model but targets cooperative AI agents rather than adversarial intrusions, compiles an agent-facing source/sink DSL rather than general provenance queries, and returns semantic feedback to the matching actor. Dynamic taint-tracking systems [11, 18, 25, 51] operate at byte or instruction granularity for vulnerability detection; AgPlane operates at object granularity, trading precision for low-overhead whole-session coverage.

eBPF enforcement and agent policy systems. BPF-LSM [42, 43] lets eBPF programs attach to security hooks and return allow/deny verdicts; AgPlane uses it as its pre-operation enforcement backend. Contemporary eBPF tools such as Tetragon, Tracee, and eBPF-PATROL [3, 10, 20] provide per-event predicates or single-channel lineage flags; AgPlane propagates multi-label information flow across channels and checks cross-event conditions. At the agent layer, programmable rails and guard agents check prompts or action trajectories [38, 49], and AgentSpec [46] enforces deterministic guards at the tool-call boundary; AgPlane complements them below the tool API, where shell-outs and subprocesses remain visible. FIDES and CaMeL [14, 15] apply typed IFC within the agent loop, which catches data-flow violations at the API boundary but cannot observe effects that escape through shell-outs, subprocesses, or compiled binaries; they also do not expose a layered authority stack that separates platform, project, and agent policy. Flume [26] provides decentralized labels with explicit capabilities but requires per-application integration rather than a compile-time authority hierarchy. AgPlane applies information-flow reasoning at the OS boundary with a layered authority model, where observation persists across context windows and cannot be circumvented by subprocess indirection.

Agent instruction files. Recent studies characterize `CLAUDE.md` and `AGENTS.md` files along dimensions of content taxonomy, maintenance practices, and task efficiency [7, 8, 29, 39]. These works classify at file or section-heading granularity. Our empirical study (§3) classifies at statement level along four axes, specifically asking which instructions require cross-event enforcement and which need project or task context to instantiate.

8 Conclusion

Agent instruction files are behavioral policies, and most of them constrain system-level actions that span multiple events. AgPlane makes these policies enforceable by compiling a compact DSL into labeled information-flow rules that run in an eBPF/BPF-LSM engine below the tool layer, and by returning semantic feedback that enables agent recovery. On a 190-trace benchmark the system achieves a 75.8% decision-compliance rate, 27–30 percentage points above prompt-filter and tool-regex baselines, while adding less than 2% end-to-end overhead. The current scope is limited to OS-observable directives (45% of the corpus); content-inspection and semantic-only directives require complementary mechanisms. By placing enforcement at the syscall boundary with agent-facing feedback, AgPlane provides a foundation for

harness-level policy that remains effective regardless of how tools, subprocesses, or sub-agents invoke system operations.

References

[1] Anthropic. 2025. Claude Code. https://docs.anthropic.com/en/docs/claude-code.

[2] Anysphere. 2025. Cursor: The AI Code Editor. https://cursor.com/.

[3] Aqua Security. 2026. Tracee: Linux Runtime Security and Forensics using eBPF. https://github.com/aquasecurity/tracee.

[4] Adam Bates, Dave Tian, Kevin R. B. Butler, and Thomas Moyer. 2015. Trustworthy Whole-System Provenance for the Linux Kernel. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, DC, 319–334. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/bates

[5] Birgitta Boeckeler. 2026. Harness Engineering for Coding Agent Users. https://martinfowler.com/articles/harness-engineering.html.

[6] Canonical Ltd. 2024. AppArmor Security Profiles. https://apparmor.net/.

[7] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. 2025. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884. https://arxiv.org/abs/2511.12884

[8] Worawalan Chatlatanagulchai, Kundjanasith Thonglek, Brittany Reid, Yutaro Kashiwa, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, and Hajimu Iida. 2025. On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code. arXiv:2509.14744. https://arxiv.org/abs/2509.14744

[9] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, et al. 2025. LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents. arXiv:2505.03574. https://arxiv.org/abs/2505.03574

[10] Cilium Project. 2026. Tetragon: eBPF-based Security Observability and Runtime Enforcement. https://tetragon.io/.

[11] James Clause, Wanchun Li, and Alessandro Orso. 2007. Dytan: A Generic Dynamic Taint Analysis Framework. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis*. Association for Computing Machinery, London, United Kingdom, 196–206. doi:10.1145/1273463.1273490

[12] Cognition. 2025. Devin: The First AI Software Engineer. https://devin.ai/.

[13] Jonathan Corbet. 2015. A seccomp overview. https://lwn.net/Articles/656307/.

[14] Manuel Costa, Boris Koepf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Beguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643. https://arxiv.org/abs/2505.23643

[15] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813. https://arxiv.org/abs/2503.18813

[16] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*. https://proceedings.neurips.cc/paper_files/paper/2024/file/97091a5177d8dc64b1da8bf3e1f6fb54-Paper-Datasets_and_Benchmarks_Track.pdf

[17] Yangruibo Ding, Siyuan Cheng, Xiaohan Wang, Yifan Song, Yiming Wang, Markus Mock, and Chenguang Wang. 2026. OctoBench: Benchmarking Scaffold-Aware Instruction Following in Repository-Grounded Agentic Coding. arXiv:2601.10343. https://arxiv.org/abs/2601.10343

[18] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2010. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. USENIX Association, Vancouver, Canada, 393–407. https://www.usenix.org/conference/osdi10/taintdroid-information-flow-tracking-system-realtime-privacy-monitoring

[19] William Findlay, David Barrera, and Anil Somayaji. 2021. BPFContain: Fixing the Soft Underbelly of Container Security. arXiv:2102.06972. https://arxiv.org/abs/2102.06972

[20] Sangam Ghimire, Nirjal Bhurtel, Roshan Sahani, and Sudan Jha. 2025. eBPF-PATROL: Protective Agent for Threat Recognition and Overreach Limitation using eBPF in Containerized and Virtualized Environments. arXiv:2511.18155. https://arxiv.org/abs/2511.18155

[21] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. Association for Computing Machinery, 79–90. doi:10.1145/3605764.3623985

[22] Invariant Labs. 2025. Invariant Guardrails Documentation. https://explorer.invariantlabs.ai/docs/guardrails/.

[23] Yuxin Jiang, Yufei Wang, Xingshan Zeng, Wanjun Zhong, Liangyou Li, Fei Mi, Lifeng Shang, Xin Jiang, Qun Liu, and Wei Wang. 2024. FollowBench: A Multi-level Fine-grained Constraints Following Benchmark for Large Language Models. arXiv:2310.20410. https://arxiv.org/abs/2310.20410

[24] Yuxuan Jiang, Meng Xu, Yiwen Song, and Xinwen Fu. 2025. AgentSight: Bridging the Semantic Gap Between Agent Intent and Low-Level Actions. arXiv:2508.02736. https://arxiv.org/abs/2508.02736

[25] Vasileios P. Kemerlis, Georgios Portokalidis, Kangkook Jee, and Angelos D. Keromytis. 2012. libdft: Practical Dynamic Data Flow Tracking for Commodity Systems. In *Proceedings of the 8th ACM SIGPLAN/SIGOPS Conference on Virtual Execution Environments*. Association for Computing Machinery, London, United Kingdom, 121–132. doi:10.1145/2151024.2151042

[26] Maxwell Krohn, Alexander Yip, Micah Brodsky, Natan Cliffer, M. Frans Kaashoek, Eddie Kohler, and Robert Morris. 2007. Information Flow Control for Standard OS Abstractions. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*. ACM, 321–334.

[27] KubeArmor Authors. 2026. KubeArmor: Cloud Native Runtime Security Enforcement System. https://kubearmor.io/.

[28] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. In *Transactions of the Association for Computational Linguistics*, Vol. 12. 157–173.

[29] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. arXiv:2601.20404. https://arxiv.org/abs/2601.20404

[30] Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, and Margo Seltzer. 2006. Provenance-Aware Storage Systems. In *2006 USENIX Annual Technical Conference (USENIX ATC 06)*. USENIX Association, Boston, MA, 43–56. https://www.usenix.org/conference/2006-usenix-annual-technical-conference/provenance-aware-storage-systems

[31] Andrew C. Myers. 1999. JFlow: Practical Mostly-Static Information Flow Control. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '99)*. ACM, 228–241.

[32] OpenAI. 2025. Codex CLI. https://github.com/openai/codex.

[33] Thomas F. J.-M. Pasquier, Xueyuan Han, Thomas Moyer, Adam Bates, Olivier Hermant, David Eyers, Jean Bacon, and Margo Seltzer. 2018. Runtime Analysis of Whole-System Provenance. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, Toronto, Canada, 1601–1616. doi:10.1145/3243734.3243776

[34] Thomas F. J.-M. Pasquier, Jatinder Singh, David Eyers, and Jean Bacon. 2017. CamFlow: Managed Data-Sharing for Cloud Services. *IEEE Transactions on Cloud Computing* 5, 3 (2017), 472–484. doi:10.1109/TCC.2015.2489211

[35] Fábio Perez and Ian Ribeiro. 2022. Ignore Previous Prompt: Attack Techniques For Language Models. arXiv:2211.09527. https://arxiv.org/abs/2211.09527

[36] Devin J. Pohly, Stephen McLaughlin, Patrick McDaniel, and Kevin Butler. 2012. Hi-Fi: Collecting High-Fidelity Whole-System Provenance. In *Proceedings of the 28th Annual Computer Security Applications Conference*. Association for Computing Machinery, Orlando, FL, 259–268. doi:10.1145/2420950.2420989

[37] Yunjia Qi, Hao Peng, Xiaozhi Wang, Amy Xin, Youfeng Liu, Bin Xu, Lei Hou, and Juanzi Li. 2025. AGENTIF: Benchmarking Instruction Following of Large Language Models in Agentic Scenarios. arXiv:2505.16944. https://arxiv.org/abs/2505.16944

[38] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 431–445. https://aclanthology.org/2023.emnlp-demo.40

[39] Helio Victor F. Santos, Vitor Costa, Joao Eduardo Montandon, and Marco Tulio Valente. 2025. Decoding the Configuration of AI Coding Agents: Insights from Claude Code Projects. arXiv:2511.09268. https://arxiv.org/abs/2511.09268

[40] Edward J. Schwartz, Thanassis Avgerinos, and David Brumley. 2010. All You Ever Wanted to Know About Dynamic Taint Analysis and Forward Symbolic Execution (but Might Have Been Afraid to Ask). In *2010 IEEE Symposium on Security and Privacy*. IEEE, 317–331. doi:10.1109/SP.2010.26

[41] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703. https://arxiv.org/abs/2504.11703

[42] KP Singh. 2019. Kernel Runtime Security Instrumentation. Linux Plumbers Conference. https://lpc.events/event/4/contributions/454/

[43] The Linux Kernel Documentation. 2021. LSM BPF Programs. https://www.kernel.org/doc/html/v5.15/bpf/bpf_lsm.html.

[44] The Linux Kernel Documentation. 2025. Landlock: Unprivileged Access Control. https://www.kernel.org/doc/html/latest/userspace-api/landlock.html.

[45] Vivek Trivedy. 2026. The Anatomy of an Agent Harness. https://www.langchain.com/blog/the-anatomy-of-an-agent-harness.

[46] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE). doi:10.1145/3744916.3764546

[47] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *Proceedings of the 13th International Conference on Learning Representations*. https://openreview.net/forum?id=OJd3ayDDoF

[48] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *19th USENIX Security Symposium (USENIX Security 10)*. USENIX Association, Washington, DC. https://www.usenix.org/conference/usenixsecurity10/capsicum-practical-capabilities-unix

[49] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. 2025. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR. https://openreview.net/forum?id=2nBcjCZrrP

[50] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*. https://openreview.net/forum?id=mXpq6ut8J3

[51] Heng Yin, Dawn Song, Manuel Egele, Christopher Kruegel, and Engin Kirda. 2007. Panorama: Capturing System-wide Information Flow for Malware Detection and Analysis. In *Proceedings of the 14th ACM Conference on Computer and Communications Security*. Association for Computing Machinery, Alexandria, VA, 116–127. doi:10.1145/1315245.1315261

[52] Nickolai Zeldovich, Silas Boyd-Wickizer, Eddie Kohler, and David Mazières. 2006. Making Information Flow Explicit in HiStar. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*. USENIX Association, 263–278.

[53] Jiongxiao Zhan, Yue Deng, Yiding He, Hongji Li, Jiaqi Li, Yibing Zhan, Wei Wang, Chaowei Xiao, and Bo Li. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*.